

Network Design & Automation Report

TechLink Solutions — Multi-Branch Network

Prepared by:

Mohamed Yasser Khalifa

Date:

June 25, 2026

Contents

AI Assistance Declaration.....	3
Project Overview.....	4
Network Desing.....	4
Topology Diagram	4
Ip Addressing Plan.....	5
WAN Point-to-Point Links	5
LAN Subnets	5
Router to Switch Links	6
Management Loopback Interfaces	6
Routing Strategy.....	7
Primary and Backup Routes	7
VLAN Design.....	8
DHCP Design.....	8
DHCP Pool Configuration	8
Day 0 Configuration	9
What Day 0 Configures:	9
What Python Automates (applied after Day 0):	9
Python Automation.....	10
Configuration Automation	10
Monitoring and Failover Logic	11
Monitoring Decision Table.....	11
Failover sequence (triggered automatically)	11
Failback sequence (triggered automatically).....	11
Conclusion.....	12

AI Assistance Declaration

This assignment was completed with the assistance of Artificial Intelligence (AI) tools. AI was used to support the development process, including providing guidance on technical concepts, assisting with the design and development of parts of the code, debugging, and improving the quality and clarity of the documentation. All final decisions regarding the system design, code structure, integration, testing, and verification were reviewed and performed by the author.

Project Overview

This document presents the network design and automation solution developed for TechLink Solutions, a multi-branch organization with sites in Cairo (HQ), Alexandria, and Giza.

The design covers full Layer 2/3 network architecture including IP addressing, VLAN segmentation, inter-VLAN routing via SVIs, redundant WAN links, and static routing with failover capability.

A Python-based automation system was developed using Netmiko to handle initial device configuration, continuous link monitoring, and automatic failover and failback without manual intervention. The solution is demonstrated via mock simulation and is designed to be deployable on real Cisco IOS devices with minimal modification.

Network Desing

Topology Diagram

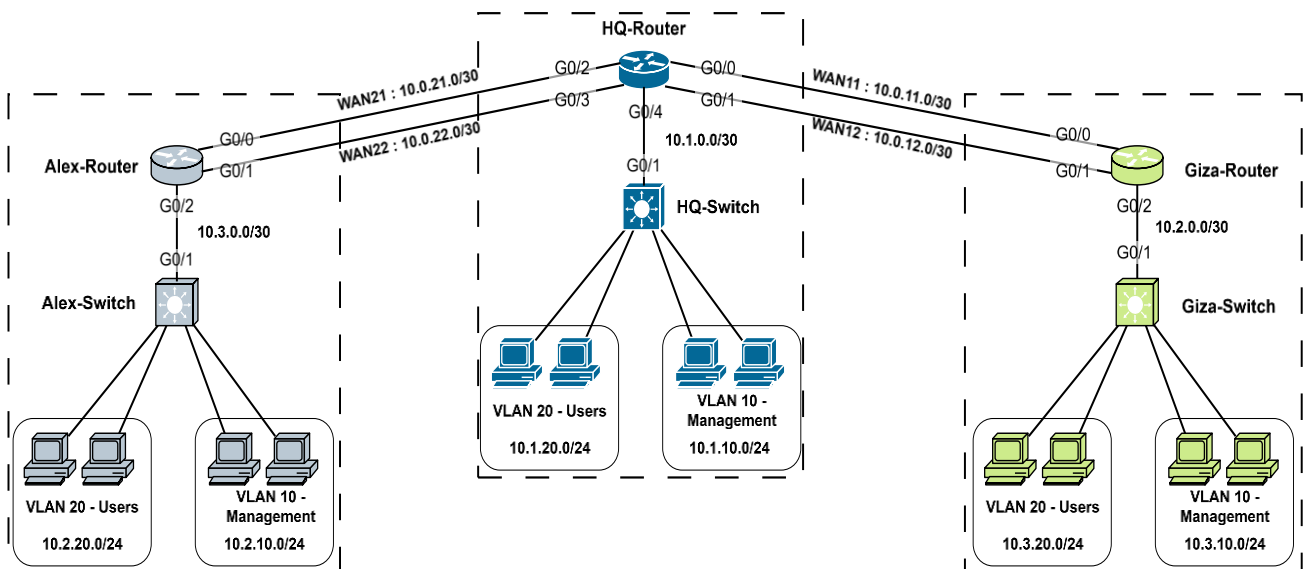


Figure 1 - Network Topology

Ip Addressing Plan

WAN Point-to-Point Links

Link	Subnet	Prefix	Router A	Interface	IP	Router B	Interface	IP
Cairo-Giza (Primary)	10.0.11.0	/30	HQ-Router	Gi0/0	10.0.11.1	Giza-Router	Gi0/0	10.0.11.2
Cairo-Giza (Backup)	10.0.12.0	/30	HQ-Router	Gi0/1	10.0.12.1	Giza-Router	Gi0/1	10.0.12.2
Cairo-Alex (Primary)	10.0.21.0	/30	HQ-Router	Gi0/2	10.0.21.1	Alex-Router	Gi0/0	10.0.21.2
Cairo-Alex (Backup)	10.0.22.0	/30	HQ-Router	Gi0/3	10.0.22.1	Alex-Router	Gi0/1	10.0.22.2

Table 1 - WAN Addressing

LAN Subnets

Site	VLAN ID	VLAN name	Subnet	Prefix	Gateway	Usable Range
Cairo	10	Management	10.1.10.0	/24	10.1.10.1	10.1.10.2 - 10.1.10.254
Cairo	20	Users	10.1.20.0	/24	10.1.20.1	10.1.20.2 - 10.1.20.254
Giza	10	Management	10.2.10.0	/24	10.2.10.1	10.2.10.2 - 10.2.10.254
Giza	20	Users	10.2.20.0	/24	10.2.20.1	10.2.20.2 - 10.2.20.254
Alexandria	10	Management	10.3.10.0	/24	10.3.10.1	10.3.10.2 - 10.3.10.254
Alexandria	20	Users	10.3.20.0	/24	10.3.20.1	10.3.20.2 - 10.3.20.254

Table 2 - LAN Subnets

Router to Switch Links

Site	Subnet	Prefix	Router Interface	IP	Switch Interface	IP
Cairo	10.1.0.0	/30	Gi0/4	10.1.0.1	Gi0/1	10.1.0.2
Giza	10.2.0.0	/30	Gi0/2	10.2.0.1	Gi0/1	10.2.0.2
Alexandria	10.3.0.0	/30	Gi0/2	10.3.0.1	Gi0/1	10.3.0.2

Table 3 - Router to Switch Links

Management Loopback Interfaces

Device	Subnet	Prefix
HQ-Router	10.1.1.1	/32
HQ-Switch	10.11.11.11	/32
Giza-Router	10.2.2.2	/32
Giza-Switch	10.22.22.22	/32
Alexandria-Router	10.3.3.3	/32
Alexandria-Switch	10.33.33.33	/32

Table 4 - Management IPs

Routing Strategy

Static routing was selected for this design for the following reasons:

- The topology is small and fixed — 3 sites, 4 WAN links, no dynamic changes expected
- Static routes give full explicit control over which path is primary and which is backup
- Failover logic is transparent and easy to demonstrate in Python automation
- No routing protocol overhead or convergence delay

Primary and Backup Routes

Rather than using floating static routes (which would handle failover automatically in IOS), this design uses Python to manage route switching explicitly. This approach was chosen because it directly satisfies the assignment requirement to "automatically update configurations if a link goes down."

Initial state: only primary static routes are configured on each router. Failover state: Python detects primary link failure, SSHs into the router, removes the primary route, and installs the backup route. Failback state: Python detects link recovery, removes the backup route, and restores the primary route.

VLAN Design

Two VLANs are deployed at each site, Inter-VLAN routing is performed by the Layer 3 switch at each site using SVIs, the router-to-switch link is a routed port (no switchport), not a trunk. The switch uses its default route to send all non-local traffic to the router.

VLAN ID	Name	Network Address	Prefix
10	Management	10.x.10.0	/24
20	Users	10.x.20.0	/24

Table 5 - VLANs

Where x = 1 for HQ, x=2 for Giza branch, and x=3 for Alexandria branch

DHCP Design

DHCP pools are configured on each Layer 3 switch, one pool per VLAN per site. This gives each site independent DHCP service that continues to operate even if the WAN link to HQ is down.

DHCP Pool Configuration

Each pool excludes addresses .1 through .10, reserving them for infrastructure. DHCP assignment begins at .11.

Site	Pool Name	Network	Gateway	DNS
Cairo	HQ_VLAN10_Management	10.1.10.0/24	10.1.10.1	8.8.8.8
Cairo	HQ_VLAN20_Users	10.1.20.0/24	10.1.20.1	8.8.8.8
Giza	Giza_VLAN10_Management	10.2.10.0/24	10.2.10.1	8.8.8.8
Giza	Giza_VLAN20_Users	10.2.20.0/24	10.2.20.1	8.8.8.8
Alexandria	Alex_VLAN10_Management	10.3.10.0/24	10.3.10.1	8.8.8.8
Alexandria	Alex_VLAN20_Users	10.3.20.0/24	10.3.20.1	8.8.8.8

Table 6 - DHCP Configuration

Day 0 Configuration

Day 0 configuration refers to the initial manual setup applied to each device via console cable before any remote automation can take place, this is a standard industry practice to bootstrap the device with the minimum configuration needed to enable SSH access.

What Day 0 Configures:

- Hostname
- IP domain name (required for SSH key generation)
- RSA key generation (2048-bit) for SSHv2
- Enable secret and local username/password
- Loopback interface (management IP)
- VTY lines for SSH-only access

What Python Automates (applied after Day 0):

- All WAN interface IPs
- LAN trunk/routed port IPs
- Static routes
- VLANs
- SVIs
- DHCP pools
- Default routes on switches

Day 0 configuration files for all 6 devices are provided as separate attachments.

Python Automation

The automation system is organized into 4 files with clear separation of concerns:

File	Purpose
config.json	Device data — IPs, credentials, interfaces, routes, VLANs, DHCP pools
configure.py	NetworkConfigurator class — pushes initial configuration to all devices
monitor.py	NetworkMonitor class — monitors WAN links and handles failover/failback
main.py	Orchestrator — loads config, calls configure, starts monitoring loop

Table 7 - Python Automation files

This structure separates data from logic. All device-specific information lives in config.json. The Python files contain only logic and read their parameters from the JSON file at runtime. Changing a device IP or adding a new VLAN requires only editing config.json — no code changes needed.

Configuration Automation

The NetworkConfigurator class establishes an SSH connection to each device using Netmiko's ConnectHandler and pushes configuration commands using send_config_set(). It handles routers and switches with the same class — the JSON file dictates what gets configured on each device.

Within the NetworkConfigurator class, methods like configure_vlan, configure_interface, and configure_static_route generate Cisco IOS commands based on the information stored in the config.json **Source of Truth**. Instead of running commands directly, these methods convert the configuration data into command lists for each branch's requirements. The generated commands are then sent to the _send_config method, which handles applying them either to a real device using Netmiko or to a simulated environment when MOCK_MODE is enabled.

Monitoring and Failover Logic

The NetworkMonitor class runs a continuous loop, checking WAN link health every 10 seconds using system ping commands via Python's subprocess module.

Monitoring Decision Table

Ping Results	Current Role	Action
Success	Primary active	Do nothing — link healthy
Success	Backup active	Failback — restore primary route
Fail	Primary active	Failover — push backup route
Fail	Backup active	Do nothing — backup already active

Table 8 - Monitoring Decision

Failover sequence (triggered automatically)

1. Ping to peer IP fails
2. Monitor detects primary link is down
3. SSH into affected router via Netmiko
4. Remove primary static route
5. Install backup static route
6. Log event with timestamp
7. Update in-memory state tracker

Failback sequence (triggered automatically)

1. Ping to peer IP recovers
2. Monitor detects primary link is back up
3. SSH into affected router via Netmiko
4. Remove backup static route
5. Restore primary static route
6. Log event with timestamp
7. Update in-memory state tracker

In a production environment, this script would run as a managed service on the NMS server, operating continuously with automatic restart on failure.

Conclusion

This document presents a complete multi-branch network design and Python automation solution for TechLink Solutions. The design satisfies all assignment requirements:

- Full network topology with 3 sites, dual redundant WAN links, and Layer 3 switching
- Structured IP addressing plan using private Class A space with logical site-based allocation
- Static routing with Python-managed failover and failback
- VLAN segmentation with SVI-based inter-VLAN routing
- DHCP services configured per site on Layer 3 switches
- Python automation covering initial device configuration, continuous monitoring, and automatic route switching
- Mock simulation mode producing realistic output logs

The automation system is production-ready in design — deploying to real hardware requires setting `MOCK_MODE = False` and ensuring Day 0 configuration has been applied to all devices. In production, the monitoring script would run as a 24/7 managed service on a dedicated NMS server.